Project Plan

Secure Password Manager

CS467 Spring 2026

Kat Loeffler, Brandon Mcconathy,

Spencer Lan, & Roman Depyak

Change Log

Who did what and when. Helps to have an audit trail IRL. The table can grow as needed.

Change	Author	Date

Introduction

As online services continue to expand, individuals are required to manage an increasing number of credentials. Without effective tools to support these users, it is challenging to maintain strong security practices, such as using long, complex passwords for each account. Consequently, many users choose to reuse passwords, create simpler passwords, or store their passwords insecurely, which increases their risk of a security breach. Password managers can offer a solution to these challenges, but users may view them as expensive, difficult to use, or lacking transparency. The goal of this project is to develop a web-based password manager that provides users with a secure and user-friendly option for managing their credentials. The main features of the project will include account creation and management, secure login and multi-factor authentication (MFA), an encrypted credential vault, and a password generation tool.

What differentiates this secure password manager from the currently available options is the focus on security, transparency, and accessibility. First, this system will use a client-side encryption model, in which sensitive data will be encrypted within the user's browser before it is ever transmitted to the server, aligning with zero-knowledge design principles. Combined with mandatory MFA and a strong, user-created master password, this password manager supports a privacy-conscious model for credential management. The system will also be developed as an open-source solution, allowing its users to inspect its security mechanisms, so that they can better understand how their data will be protected. Finally, the system will be freely available to the public to promote accessibility for all incomes. Through the combination of security, transparency, and accessibility, this project seeks to provide a practical and usable alternative to existing password managers.

Problem Statement

As the number of digital shops and services offered online grows, so do the number of online accounts that modern users have to manage. A survey conducted by NordPass in 2024 found

that, on average, individuals handle 168 passwords for personal accounts and 87 for business-related accounts [1]. Without some form of password management, this volume of passwords can become difficult to sustain, while maintaining good security practices. Subsequently, the Pew Research Center found that nearly seven-in-ten U.S. adults are overwhelmed by the number of passwords they have to manage, and nearly half say they feel anxious regarding the security of those passwords [2]. While the Cybersecurity and Infrastructure Security Agency (CISA) recommends using a long, random, unique password for every account, many users struggle to remember or organize their credentials safely, which leads to the use of unsafe passwords and insecure storage methods [3].

In response to these stresses individuals resort to using less secure methods to make password management less burdensome. For example, people may reuse passwords or choose simpler passwords when creating new credentials. Nearly half of U.S. adults admit to prioritizing easy-to-remember passwords over more secure passwords [2], and greater than 78% of people globally admit to reusing passwords across accounts [1]. This practice can put multiple accounts at risk, because if one account is breached, then all accounts with the reused password are vulnerable. Further, over half of all U.S. adults report using less secure password management methods, including writing them down, memorization, or browser storage [2]. Password managers can offer users solutions to these issues, including password generation, secure storage, and synchronization across devices. However, existing password managers may also create barriers for users by being expensive, closed-source, or too complex. In 2024, only 36% of individuals reported using a password manager and among non-users, affordability, security, and perceived complexity are reported as major deterrents [4].

These challenges affect a very broad population of internet users, particularly students, families, older adults, and budget-conscious individuals who want a free and open solution that balances usability and security. Additionally, those who despite transparency into how their credentials are protected would also benefit from an open-source password manager. This highlights the need for a trustworthy, affordable, and transparent system for password management that reduces stress and mental load, while promoting strong security practices.

To address these issues, our team proposes the development of a secure, web-based password manager designed to balance usability, accessibility, and security. Our password manager will store credentials in an encrypted vault, accessed by a master password and multi-factor authentication (MFA) to increase security, based on the best practices laid out by CISA [5]. In addition to secure vault storage, our password manager will also offer generation of strong, unique passwords. The application will be hosted securely in the cloud, and will be available to users through a simple-to-use interface. Finally, we will provide documentation and user guides for the application for transparency.

Requirements

To develop a secure, user-friendly, and transparent web-based password manager, it is important to first define the requirements of the application to ensure that the final product will

meet the needs of its users. The purpose of these requirements is to provide a basis for the design and development of the password manager, as well as a foundation for testing and validation. Below is a listing of the most important features of the password management system, including both functional and non-functional.

Functional Requirements

- The system should allow users to create an account.
- The system should require users to create a single, strong master password to unlock their vault.
- The system should prompt users, upon first login, to register at least one MFA method, either SMS or email-based one-time-password (OTP).
- The system should allow users to securely log in and log out of their account.
- The system should require successful authentication and MFA verification before the user can unlock and access encrypted vault data.
- The system should allow users to securely create, retrieve, update, and delete credentials.
- The system should store credentials in an encrypted vault.
- The system should offer a password generation feature that produces strong, unique, and customizable passwords.
- The system should provide a clean interface with clear organization and a search function for finding credentials.
- The system should provide user guides to enable troubleshooting.
- The system should include documentation within the repository and be sufficient for a new developer to install, configure, and run the application.

Non-functional Requirements

- The system should utilize industry-standard encryption practices to protect sensitive vault data during transmission and storage.
- The system should encrypt sensitive data before it is stored to minimize exposure of plaintext credentials.
- The system should be designed to minimize unnecessary exposure of plaintext credentials in both client and server components.
- The system should maintain high availability, targeting an uptime of 99.9% for accessing credentials.
- The system should provide a user experience that allows a new user to register and store their first password within five minutes without consulting a user guide.
- The system should be developed as an open-source solution to promote transparency and allow users to inspect security mechanisms.

Design / Architecture

The structure of the secure password manager is designed to support the functional and non-functional requirements outlined above, with a focus on security, usability, and transparency. The system will follow a client-server architecture. Users will interact with a web-based frontend built with React, which will also be responsible for performing client-side encryption of user credentials before they are transmitted. The React frontend will communicate with a Flask backend through API requests. The backend will be responsible for authentication, MFA, and credential storage in a secure database. The design of the password manager will use clean, minimalist interfaces with consistent design elements to make sure that the system is simple to use. By encrypting user credentials in the frontend, the system also ensures that sensitive data remains encrypted during transmission and at rest. See Figure 1 below for a diagram of the system's architecture.

The following are the high-level components that will make up the secure password manager:

1. **Frontend/User Interface (React)**
    - This component is the part of the system that the user will interact with directly through the web application. It will be implemented using React and will provide screens and workflows for registration, login and MFA, a vault dashboard with search functionality, an add/edit credential page with inbuilt password generator, as well as a documentation/help page. See Figure 2 below for a visualization of the user interface layout. In addition to managing the UI, this component is responsible for handling client-side encryption and decryption of sensitive vault data. Credential information is encrypted within the browser before being transmitted to the backend and decrypted only after successful authentication. The frontend also manages application state and communicates with the backend through API requests to retrieve and store encrypted data.
    - The requirements supported by this component include the following:
        - User account creation
        - Login and logout functionality
        - Viewing and organizing stored credentials
        - Password generation
        - Search and usability
        - Client-side encryption and secure handling of credential data

2. **Backend Application (Flask)**
    - This component is responsible for API orchestration, routing and user session management. The backend will bridge the React frontend and the database, processing incoming API requests and ensuring that only users who are authenticated can access or modify their stored data. The backend coordinates authentication workflows, including login and MFA, and it does not directly handle encryption of credential data. Instead, it will receive and store data that has

already been encrypted in the frontend to make sure that plaintext credentials are not required by the server.
    - The requirements supported by this component include the following:
        - Log in and log out of accounts securely
        - User account registration and management
        - Session or token-based authentication (e.g., Flask sessions or JWTs)
        - Multi-factor authentication (OTP generation and verification)
        - Secure API routing and access control for protected resources

3. **Authentication & MFA Service (Email OTP)**
    - This component is responsible for supporting MFA through the use of one-time passwords (OTPs) delivered via email. It is integrated with the backend and is triggered after a user successfully submits their login credentials. Upon login, it generates and sends an OTP to the user's email and then validates the OTP provided by the user during the authentication process. This component relies on a third-party email service or library to deliver the OTP messages.
    - The requirements supported by this component include the following:
        - Send valid OTP to the correct user's email
        - Send the email in a reasonable amount of time
        - Correctly validate the OTP when the user enters it to login
        - Communicate OTP validation to the backend to allow or deny the user to login

4. **Encryption Service (CryptoJS)**
    - This component is the part of the system that is responsible for encrypting and decrypting the user's sensitive data within the frontend before sending it to the backend. It is also responsible for decrypting the encrypted data that is sent from the server to the client. It supports the application's client-side encryption model by ensuring that stored credentials are protected before being sent to the server.
    - The requirements supported by this component include the following:
        - Encrypt credential data in the frontend before it is transmitted to the backend
        - Decrypt the encrypted data in the frontend after it is retrieved from the backend
        - Minimize backend exposure to unencrypted credentials

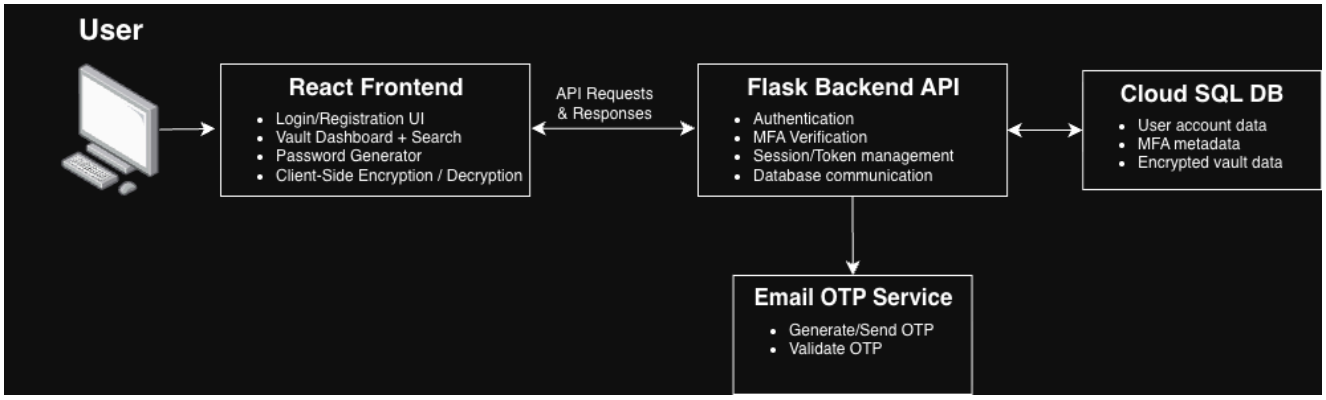
5. **Database/Storage Layer (SQLite)**
    - This component is the part of the system that stores application data, including user account records, encrypted vault data, and MFA-related information. It interacts with the Flask backend to save, retrieve, update, and delete data as needed by users. In the deployed system, this layer will use a managed relational database such as MySQL through Google Cloud SQL, while SQLite may be used during local development and testing.
    - The requirements supported by this component include the following:

        - Stores user account data.
        - Stores encrypted data.
        - Support CRUD operations on data through the backend.

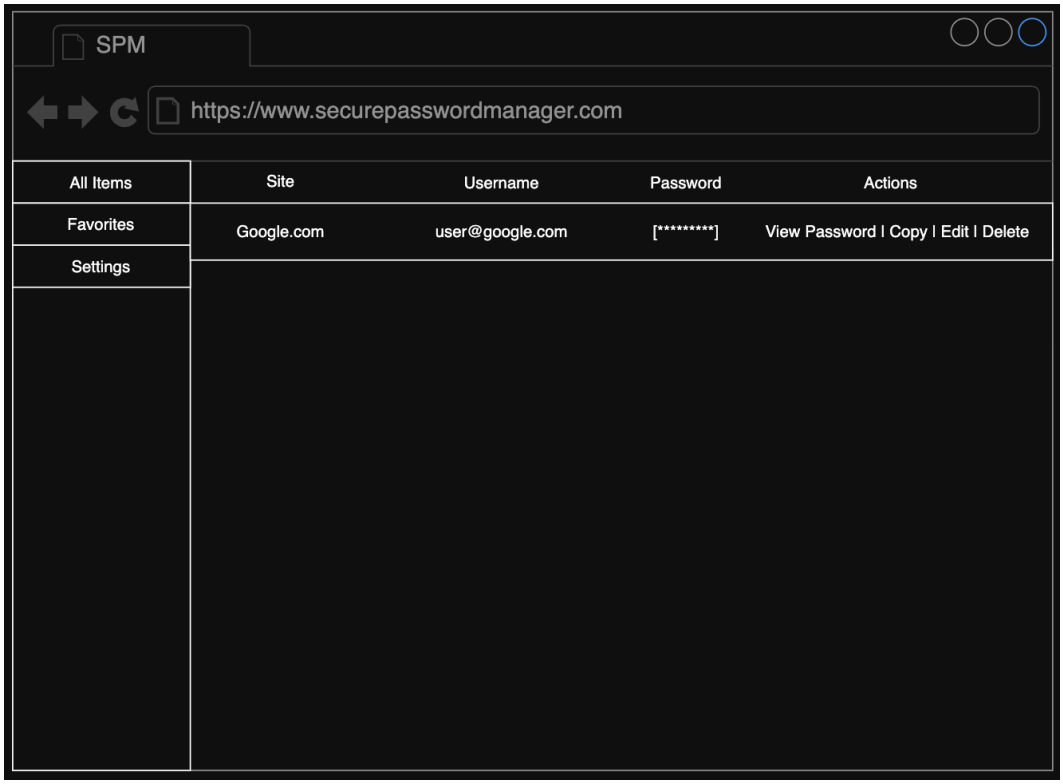
6. **Cloud Infrastructure/Deployment (Google Cloud)**
    - This component is the part of the system that is responsible for hosting and deploying the application so that authorized users can access the webpage over the internet. The system will use Google Cloud services to host the frontend and backend application components, with data stored in a managed cloud database. This deployment is intended to support high-availability.
    - The requirements supported by this component include the following:
        - Authorized users are able to access the webpage from any internet connection
        - All services (frontend, backend, and database) are hosted through the cloud provider
        - The system should maintain high availability, targeting an uptime of 99.9% for accessing credentials

Visuals

- **Figure 1**: High-level architecture diagram demonstrating the React frontend, Flask backend, Cloud SQL database, and email-based OTP service.



- **Figure 2**: Mockup of user dashboard.



Technologies

Languages

- Python
- HTML/CSS
- SQL
- JavaScript

Frameworks/Libraries

- Flask
- Flask-Login
- Flask-WTF
- SQLAlchemy
- cryptography
- PyOTP
- CryptoJS
- React

Database/Storage

- SQLite (for local development)
- PostgreSQL (for production deployment via Google Cloud SQL)

Development Tools

- Git
- GitHub
- GitHub Projects

Hosting/Deployment

- Google Cloud

One important technology that we have chosen is Flask. Our group has chosen Flask as our backend framework because its minimalist design allows for us to balance this project's stringent need for high security along with our group member's skill set. Unlike a more comprehensive framework such as Django, Flask provides essentials such as routing, request handling, and templating, but does not include non-essentials that may be used in a more complex or less security focused project. As Flask is more minimalistic as compared to a framework like Django, this is a pro for our security focused project as it should minimize vulnerabilities and bloat in our codebase. The goal of our password manager is zero knowledge, to best project user data. Having more control with a framework such as Flask will better help us achieve that goal as compared to a more complex framework such as Django because we will have more control over how data moves from our API to our SQLite database. Flask is also a good choice because our group is familiar with Python and Flask has a shallow learning curve. One con of Flask is that it does not have built in security features such as CSRF protection or a built in sophisticated user authentication system. This means that Roman and Brandon will need to manually implement and configure extensions such as Flask SQLAlchemy for database integrity. This means that we will have to put in more effort with Flask as opposed to another framework, but it will allow us to gain a deeper understanding of the security measures we are implementing and give us more control as to the security measures of the project. Those are the main pros and cons of choosing Flask as compared to another framework. Our team's weakness is that none of us have ever worked on a project that focuses on security before and Flask offers less built in or out of the box security, but this will allow us to learn more than if we had used a framework with built in security.

Important Resources (if any)

- Email-based OTP service for delivering MFA codes

Development Plan

- **Kat Loeffler**
- **Progress Report #1**
    - **Task:** Research client-side encryption approaches and evaluate suitable JavaScript cryptography libraries (e.g., CryptoJS)
        - **Time estimate:** 4 hours
        - **Dependencies/Resources/Risks:** Risk of choosing a library that is difficult to integrate or lacks required features
    - **Task:** Design high-level encryption workflow, including how vault data will be encrypted before transmission and decrypted after authentication
        - **Time estimate:** 6 hours
        - **Dependencies/Resources/Risks:** Requires alignment with backend authentication flow; risk of design changes later
    - **Task:** Set up React project structure for handling secure state and API communication
        - **Time estimate:** 6 hours
        - **Dependencies/Resources/Risks:** Dependent on team decisions regarding project setup and tooling
    - **Task:** Implement initial encryption and decryption utility functions for testing basic functionality
        - **Time estimate:** 6 hours
        - **Dependencies/Resources/Risks:** Risk of incorrect implementation leading to later rework
- **Progress Report #2**
    - **Task:** Develop full encryptVault and decryptVault functionality for handling credential data
        - **Time estimate:** 10 hours
        - **Dependencies/Resources/Risks:** Requires finalized data structure for credentials
    - **Task:** Implement key derivation logic based on the user's master password
        - **Time estimate:** 8 hours
        - **Dependencies/Resources/Risks:** Risk of improper key handling affecting security
    - **Task:** Integrate encryption logic with backend API to ensure only encrypted data is transmitted and stored
        - **Time estimate:** 8 hours
        - **Dependencies/Resources/Risks:** Strong dependency on backend API readiness; risk of mismatched data formats
    - **Task:** Implement secure frontend state management for handling encrypted and decrypted vault data
        - **Time estimate:** 6 hours
        - **Dependencies/Resources/Risks:** Risk of exposing sensitive data in memory if not handled carefully

- **Progress Report #3**
    - **Task:** Build and integrate password generator functionality within the frontend
        - **Time estimate:** 6 hours
        - **Dependencies/Resources/Risks:** Must align with UI requirements and backend expectations if applicable
    - **Task:** Improve encryption handling for edge cases (empty fields, updates, partial edits)
        - **Time estimate:** 6 hours
        - **Dependencies/Resources/Risks:** Risk of data corruption if edge cases are not handled properly
    - **Task:** Implement error handling and validation for encryption/decryption failures
        - **Time estimate:** 5 hours
        - **Dependencies/Resources/Risks:** Requires testing across multiple scenarios
    - **Task:** Optimize performance and responsiveness of encryption operations in the browser
        - **Time estimate:** 5 hours
        - **Dependencies/Resources/Risks:** Risk of performance issues with large datasets
- **Project Archive - Final**
    - **Task:** Conduct end-to-end testing of encryption workflow across login, MFA, vault access, and credential management
        - **Time estimate:** 6 hours
        - **Dependencies/Resources/Risks:** Requires full system integration
    - **Task:** Debug and resolve issues related to frontend-backend encryption integration
        - **Time estimate:** 6 hours
        - **Dependencies/Resources/Risks:** Dependent on stability of backend and database
    - **Task:** Assist in writing technical documentation describing encryption design and usage
        - **Time estimate:** 4 hours
        - **Dependencies/Resources/Risks:** Requires finalized implementation
    - **Task:** Participate in final system testing and refinement of security-related features
        - **Time estimate:** 4 hours
        - **Dependencies/Resources/Risks:** Time constraints near deadline
- **Total Hours: 90 hours**

- **Brandon Mcconathy**
- **Progress Report #1**
    - **Task:** Set up development environment, install Flask, dependencies, and configure project structure.

        - **Time estimate:** 6
        - **Risks:** Python/Flask compatibility issues, environment setup errors.
    - **Task:** Learn and research Flask authentication methods, MFA concepts, and JWT/session handling.
        - **Time estimate:** 8
        - **Risks:** Learning curve, unclear best practices for security.
    - **Task:** Implement basic Flask routes for user registration and login functionality.
        - **Time estimate:** 10
        - **Risks:** Route structure errors, database connection issues.
- **Progress Report #2**
    - **Task:** Research and learn Python OTP libraries.
        - **Time estimate:** 8
        - **Risks:** Learning curve, best practices.
    - **Task:** Implement OTP system for sending verification codes.
        - **Time estimate:** 10
        - **Risks:** Third-party API reliability, email delivery delays, API key configuration.
    - **Task:** Implement secure storage and expiration handling for OTP codes.
        - **Time estimate:** 6
        - **Risks:** Timing issues, improper expiration logic.
- **Progress Report #3**
    - **Task:** Implement JWT token generation and verification OR configure Flask session-based authentication.
        - **Time estimate:** 10
        - **Risks:** Token security risks, improper session handling.
    - **Task:** Integrate authentication flow end-to-end.
        - **Time estimate:** 8
        - **Risks:** Integration bugs between components
    - **Task:** Conduct testing for authentication and MFA flow.
        - **Time estimate:** 8
        - **Risks:** Edge cases, incomplete test coverage.
- **Project Archive - Final**
    - **Task:** Debug, optimize, and finalize authentication and MFA system.
        - **Time estimate:** 6
        - **Risks:** Hidden bugs, time constraints.
    - **Task:** Assist with deployment and ensure secure configuration in production environment..
        - **Time estimate:** 6
        - **Risks:** Deployment environment differences, security misconfigurations.
- **Total Hours: 86**

- **Spencer Lan**
- **Progress Report #1**

    - Design wireframes for the core user flow (look into how to structure Landing, Login and Dashboard pages).
        - Time estimate in hours: 15 hours
        - Major Dependencies/Resources/Risks (if any)
            - Requires understanding the basic structure of the security architecture from the backend to ensure the UI elements reflect and represent the correct data items.
    - Set up the React project environment and implement foundational layout components such as the Navbar, Sidebar, and authentication states.
        - Time estimate in hours: 10 hours
        - Major Dependencies/Resources/Risks (if any)
            - Need to research React/Node environment setup and plan out how to structure directories.
            - Research how to store tokens and active sessions.
- **Progress Report #2**
    - Build the dashboard shell, making sure we have a clean interface for showing, filtering and categorizing stored credentials.
        - Time estimate in hours: 15 hours
        - Major Dependencies/Resources/Risks (if any)
            - Dependent on the backend API being ready to return mocked data for UI testing (temp fields for user data, etc.)
    - Implement searching and filtering logic to allow users to find passwords as they type.
        - Time estimate in hours: 12 hours
        - Major Dependencies/Resources/Risks (if any)
            - Unsure if there will be any performance risks with the number of passwords we'll store. There shouldn't be a huge number with our tests, but scalability might be a concern.
- **Progress Report #3**
    - Integrate Frontend with Backend endpoints for fetching and updating saved password data.
        - Time estimate in hours: 15 hours
        - Major Dependencies/Resources/Risks (if any)
            - Need to manage updated data (Availability) so that the UI updates immediately after changing a password.
            - Handling of errors for any API calls that don't work properly.
    - Refine the way users add passwords, streamline UI/UX for new users to easily utilize the application.
        - Time estimate in hours: 10 hours
- **Project Archive - Final**
    - E2E Testing to verify that users can complete all CRUD operations, register, and store a password easily.
        - Time estimate in hours: 8 hours
        - Major Dependencies/Resources/Risks (if any)

            - If there are too many steps or unreasonable actions to complete all needed functionality, we may need to change UI elements.
    - Final polish of UI and see if there is additional time for extra features.
        - Time estimate in hours: 7 hours
        - Major Dependencies/Resources/Risks (if any)
            - Styling and CSS have to be consistent, may need to adjust based on screen type and size (should mostly be web based).
- **Total Hours: ~92 Hours**

- **Roman Depyak**
- **Progress Report #1**
    - High-level Requirement/Story/Feature/Task - Set up backend development environment (which includes Flask, CORS, dependencies, and the project structure).
        - 6 hours
        - Learning setup tools and environment inconsistencies. Adding Flask-CORS will allow for fetching data, as I believe Flask blocks requests from a React frontend running on a different port by default.
    - High-level Requirement/Story/Feature/Task - Design and implement the SQLite database schema(for example users and vault tables) & Flask-Migrate.
        - 6 hours
        - The schema has to align with authentication and encryption requirements. If Flask-Migrate is not properly implemented and the group needs to make modifications such as storing a salt with the encrypted data then I'll need to drop and recreate the SQLite database. If I use Flask-Migrate I can change the schema without losing any test data.
    - Implement database connection utilities and initialization scripts.
        - 4 hours
        - If there are errors in schema design they could affect the entire backend and/or the entire project. I have to make sure the schema is well thought out so that each team member's portion of the project will work together as a whole.
    - Create the initial CRUD endpoints (such as POST and GET) & perform basic input validation (see if JSON body is missing a required field).
        - 9 hours
        - If the endpoints don't align between the frontend and backend the entire project won't work. If the frontend sends an improper request to the backend and the request isn't properly validated then I won't know if the bug is with my backend code or their frontend code. Basic validation will help clear up the ambiguity.
    - Test endpoints using Postman
        - 3 hours
        - Thorough testing requires that the backend routes are stable.

- **Progress Report #2**
    - Implement update and delete endpoints for vault entries.
        - 5 hours
        - I have to maintain secure user based filtering.
    - Integrate JWT authentication into all protected routes
        - 6 hours
        - I have to coordinate & communicate with Brandon so that the authentication works as intended.
    - Implement user specific data isolation (to make sure that a user only has access to their vault and not someone else's).
        - 5 hours
        - I have to make sure that this step is implemented correctly because this is the most important aspect of this project. If a user is able to access someone else's data then the Secure Password Manager does not accomplish its number one duty.
    - Collaborate with Kat and Spencer to ensure that the backend and the client are compatible with one another and function as intended.
        - 5 hours
        - If there are any misaligned requests or response formats then the client and backend will not work with one another.
    - Add error handling and review input validation to endpoints.
        - 4 hours
        - Incorrectly implemented or lack of validation could lead to security vulnerabilities.
    - Add centralized logging
        - 3 hours
        - When we deploy to Google Cloud we likely won't have a terminal window to look for errors. If we implement a Python logging module now then when we deploy we can see why a login failed in production and we won't have to guess as to why stuff isn't working or us having to rely on a generic error message.
- **Progress Report #3**
    - Prepare backend for deployment (environment variables and configurations).
        - 5 hours
        - Errors or misconfigurations in environment variables
    - Deploy backend to Google Cloud
        - 6 hours
        - Deployment errors and platform limitations
    - Configure production server using Gunicorn
        - 4 hours
        - Incorrect server startup
    - Perform end to end testing with deployed backend
        - 5 hours
        - Integration issues between frontend and backend

    - Optimize database queries and API performance
        - 5 hours
        - Performance bottlenecks under load
- **Project Archive - Final**
    - Write thorough API documentation (such as endpoints, request and response formats).
        - 6 hours
        - Documentation has to match the final implementation
    - Create developer setup guide (installation and running)
        - 5 hours
        - The setup guide has to be clear and complete.
    - Write User Guide (how to use backend features via UI).
        - 4 hours
        - I have to ensure that the frontend UI functions as intended with my backend features
    - Final bug fixes and code cleanup
        - 5 hours
        - Unforeseen last minute problems
    - Organize GitHub repo (README, file structure, comments)
        - 5 hours
        - Poor organization
- **Total Hours: 100**

Conclusion

References

[1] Ramirez, S. (2025, September 23). *Password manager statistics 2026: Trends & growth insights*. SQ Magazine. https://sqmagazine.co.uk/password-manager-statistics/

[2] Pew Research Center. (2023, October 18). *How Americans view data privacy: The role of technology companies, AI and regulation—plus personal experiences with data breaches, passwords, cybersecurity and privacy policies*. https://www.pewresearch.org/internet/2023/10/18/how-americans-view-data-privacy/

[3] Cybersecurity and Infrastructure Security Agency. (n.d.). *Use strong passwords*. https://www.cisa.gov/secure-our-world/use-strong-passwords

[4] Frew, P., & Petrino, G. (2025, July 22). *2024 password manager industry report and statistics*. Security.org. https://www.security.org/digital-safety/password-manager-annual-report/

[5] Cybersecurity and Infrastructure Security Agency. (n.d.). *Multifactor authentication*. https://www.cisa.gov/MFA